

CSCB58 Lab 5

Part A: Control Flows

Introduction

Last week, we built a simple program in MIPS assembly and became familiar with MARS simulator. This week, we will write some programs in MIPS assembly using branches and loops. You may also find these techniques useful for your final project.

This week's lab assumes that you completed all the material from last week's lab. If you did not get the programs working last week, finish them before this lab.

[TASK] Document your code with comments, and write your name at the top of your submissions.

[TASK] Do not forget to terminate the program correctly using syscall 10.

Compute the Number of Solutions to a Quadratic Equation

Let a , b , c and x be some integers. Consider the quadratic equation: $ax^2 + bx + c = 0$. The number of real solutions for this equation can be found using the discriminant Δ :

$$\Delta = b^2 - 4ac.$$

If $\Delta < 0$ there are zero real solutions for x , if $\Delta = 0$ there is one solution, if and $\Delta > 0$ there are two.

Write an assembly program that receives the coefficients a , b , c in registers $\$t5$, $\$t6$, $\$t7$ (respectively) and computes the number of solutions. At the beginning of your program, initialize these registers to some values. When your program ends, $\$t0$ should contain the number of solutions: 0, 1, or 2. You can assume results fit in 32 bits. **Test your program well!**

Note it is fine to overwrite the registers we used for input (or output) during the run of your program, as long as at the beginning you take it from the right registers, and the end the result is stored in the right register.

[TASK] Submit a file `lab05a.asm` that implements the above. Make sure your code is well-documented.

[TASK] Write your name and UTorID at the top of the file as a comment!

Use Euclid's Algorithm to Compute Greatest Common Divisor

Let a and b be two positive, non-zero integers. Their *greatest common divisor* is the largest positive, nonzero integer that divides both a and b without leaving a remainder. For example if $a = 46$ and $b = 20$ their greatest common divisor is $\gcd(46, 20) = 2$ since $46/2 = 23$ and $20/2 = 10$ but no larger integer divides both. Other examples: $\gcd(60, 15) = 15$, $\gcd(16, 60) = 4$, $\gcd(60, 48) = 12$, $\gcd(25, 64) = 1$.

Euclid provided a simple iterative algorithm to compute the GCD, given here in Python:

```
while a != b:
    if a > b:
        a = a - b
```

```
else:
    b = b - a
```

At the end of the run, a will contain the GCD.

Implement Euclid's algorithm in assembly. Your program will first initialize `$t8` and `$t9` to two positive numbers, and then implement the algorithm above. When your program ends, the GCD of `$t8` and `$t9` should be stored in `$t0`.

[TASK] Submit a file `lab05b.asm` that implements the above. Make sure your code is well-documented.

[TASK] Write your name and UTorID at the top of the file as a comment!

Hints:

- First test the algorithm in a high level language (e.g., Python) so you can understand how it works.
- You will need a loop, and also some conditionals. This means several branches, jumps, and labels.

Summary of tasks

1. Write a program `lab05a.asm` that computes the number of solutions to a quadratic equation.
2. Write a program `lab05b.asm` that computes the GCD using Euclid's algorithm.
3. Make sure to document your assembly code using comments, and to write your name at the top.
4. Submit the programs to Quercus as a zip file. **Do so before the deadline!**
5. Demonstrate your solutions to your TA.

Part B: Arrays and Functions

Introduction

This week, we will write some programs with arrays and functions as well as recursion. You may also find these techniques useful for your project.

This week's lab assumes that you completed all the material from last week's lab. If you did not get the programs working last week, finish them before this lab.

[TASK] Document your code with comments, and write your name at the top of your submissions.

[TASK] Do not forget to terminate the program correctly using `syscall 10`.

Arrays

As we discussed in the lecture, arrays are declared in the `.data` section of the assembly code. For example, the following code fragment declares two arrays of 7 integers. The first one is initialize to some numbers, and the second one has 7 integers all initialized to 0. The variable `LEN` stores the length of the arrays.

```
.data
A:    .word    5, 8, -3, 4, -7, 2, 33
B:    .word    0:7
LEN:  .word    7  # length of the arrays
```

To access the elements of an array, we need to perform memory access instructions such as `lw` and `sw`. To access an element we must first compute its memory address. The basic formula to compute the memory address of an element is `base + offset`, where `base` is the address of the first element of the array (the label `A`), and `offset` is the index of the element multiplied by the size in bytes of each element of the array. Index of arrays is zero-based: it is always between 0 and array length minus 1. For example, the value of `A[4]` is -7, and the memory address for `A[4]` is `A + 4*4`. See the lectures for more details.

Write a program code that iterates through `A`, multiplies each element by a factor that depends on whether the number is odd or even, then stores the result in the equivalent element of `B` (hint: you can use `andi` with 1 to check if an element is odd). Your program should take the length of the arrays from the variable `LEN`, and should work for any length of an array. Even numbers in `A` are multiplied by 10, while odd numbers in `A` are multiplied by 5.

In Python this would be something like: `B[i] = A[i] * (10 if (A[i] % 2) == 0 else 5)`.

[TASK] Save your code as `lab05c.asm` and submit to Quercus with the other programs in a zip file.

Function Calls

Just like any high level programming language, separating code into well defined procedures or functions is an important idea. Conceptually, making function call is actually simple: we need to “jump” to another portion of code (the function body) then start executing the instructions in the function body. When we reach the end of that function, another “jump” is needed to go back to the caller.

Passing arguments and return values can be done in many different ways, depending on the *calling convention*. In this lab we are going to use two different calling convention: the simple stack-based calling convention we learned in the lecture, and the register-based calling convention where arguments are passed using `$a0` to `$a3` (first argument in `$a0`, second in `$a1` and so on) and return value is passed using `$v0`.

Write an assembly program to implement the pseudocode listed below, subject to the following requirements:

- The function `do_addition` should use the register-based calling convention.
- The function `nsolutions` should use the stack-based calling convention.
- You will need to explain parts of your code to your TA and convince them you are using the correct calling convention.
- We are not going to be strict about whitespace as long as you achieve the needed important goals. If in doubt, follow the Python code.

[TASK] Save your code as `lab05d.asm` and submit to Quercus with the other programs in a zip file.

The code (in Python) you need to implement in assembly:

```
# in main
A = input("Enter a value for A")  # read A
B = input("Enter a value for B")  # read B
C = input("Enter a value for C")  # read C
```

```

print("Before function")          # this might be useful when debugging
print("A + B + C = ", do_addition(A, B, C))
print("Num solutions for Ax^2 + Bx + C is = ", n_solutions(A, B, C))

# Add the numbers
def do_addition(A, B, C):
    return A + B + C

# Compute number of solutions to quadratic
def n_solutions(A, B, C):
    delta = B*B - 4*A*C
    if delta > 0:
        return 2
    if delta == 0:
        return 1
    return 0

```

Recursion

Things get more interesting when we have multi-level function calls, especially when you are implementing a recursive function. Since the return address is automatically stored in the `$ra` register when `jal` is executed, when a function calls another function, the content of `$ra` will be overwritten and the calling function's return address will be forgotten. To avoid this, we need to remember the return address of each level of function somewhere, namely, the stack. We can access the stack using the stack pointer value stored in register `$sp`.

To push a word onto the stack, you can do:

```

addi $sp, $sp, -4    # move the stack pointer to increase stack size
sw $r, 0($sp)        # put the value in $r on the allocated space

```

To pop a word from the stack:

```

lw $r, 0($sp)        # load the word at the top of the stack
addi $sp, $sp, 4      # decrease the size of the stack

```

Other things also need to be remembered on and passed through the stack, such as the argument passed to the function being called, the return value of a function call, and the temporary values which need to be used after returning from the recursive call. The orders of the pushes and pops need to be design carefully so that you are always correctly passing and restoring the value that you expect.

Create a new file named `lab05e.asm` and implement in assembly the following piece of pseudocode, which involves a recursive function `mystery`. This task could take longer time than others to finish, so you might want to start working on it earlier than usual. You may refer to the function examples in class for explanations of how to deal with the stack and functions.

```

# in main
def main():
    n = input("Enter a number: ")
    print("The result is:", mystery(n))

```

```
# Compute mystery equation
def mystery(n):
    if n == 0:
        return 0
    return mystery(n-1) + 2*n - 1
```

Once you get it to work, be ready to answer questions about how your code works. In particular, try to break your code by passing a very large input value n . How large n can be?

[TASK] Save your code as `lab05e.asm`, zip it with the other programs and submit to Quercus.

Evaluation

As always, marks are based not only on submitted work but also on oral examination by TA. You need to be able to make reasonable explanations about any details to the TA.

Part A:		
Solution	Submitting everything on time	1 mark
	Code documentation (comments)	1 mark
	Number of solutions	2 marks
	GCD	1 mark
Part B:		
Understanding	Arrays	1 mark
	Functions	1 marks
	Recursion	2 mark
	TA oral score	1 to 4

Final lab marks (up to 9) are determined by multiplying your solution subtotal by the oral score (1–4) and dividing by 4:

$$\text{total} = \text{solution marks} \times \frac{\text{oral score}}{4}$$